ASCII Dungeon Generator

Project Reference Document

1. Project purpose

This project is an educational exercise in learning JavaScript through the development of a basic ASCII roguelike dungeon generator.

The longer-term objective is to develop a playable, turn-based dungeon environment with procedurally generated levels, a player character, movement, collision detection, and eventually additional roguelike mechanics.

The project is also intended to develop a practical understanding of JavaScript programming. Understanding the code is as important as getting it to work.

The project should therefore develop incrementally. New functionality should not be introduced until the existing functionality is understood.

2. Development philosophy

The following principles should guide all development discussions.

Incremental instruction

- Introduce concepts in small, manageable steps.
- Explain the reasoning behind design decisions, not merely the instructions for implementing them.
- Avoid assuming knowledge of JavaScript features that have not yet been introduced.
- Do not advance through several stages of development in a single response.
- Encourage experimentation and testing after meaningful changes.

Understanding before implementation

The objective is not simply to produce working code. The user should understand what the code does, why it is structured as it is, and how its individual components interact.

When appropriate, use pseudocode, conceptual explanations, and small exercises before introducing actual implementation.

Preserve existing code

- Never rename existing variables, functions, objects, or other established identifiers when revising code.
- Preserve established naming conventions.

- When modifying existing code, work from the actual code supplied by the user rather than reconstructing it from memory.
- Avoid unnecessary refactoring or abstraction.
- Specify precisely which code should be replaced and where. Do not give ambiguous replacement instructions.

The user sometimes anticipates upcoming implementation steps as a way of testing their understanding. Allow this and evaluate the proposed approach before providing further instructions.

Changes must solve the problems they are intended to solve

This is an especially important development rule.

A code change that fails to fix its intended behavior must be reverted completely before another solution is attempted.

Do not retain failed changes as groundwork for subsequent attempts. Do not accumulate speculative conditions, abandoned diagnostics, or compensating logic.

The development objective is a minimal implementation of generalized rules, not an accumulation of exceptions.

Every modification should:

1. Address a specific, identified problem.
2. Be tested against the behavior it is intended to correct.
3. Be reverted if it fails to produce the intended result.

A change should not be considered successful merely because it does not break other established behavior. It must actually solve the problem it was introduced to solve.

When a change is successful, preserve it and avoid disturbing it during subsequent modifications.

3. Current project architecture

The project uses separate JavaScript modules to distinguish dungeon construction, player behavior, collision detection, and rendering.

File	Responsibility
`index.html`	Basic webpage structure and entry point
`style.css`	Display styling
`main.js`	Application coordination and dungeon display
`dungeon_gen.js`	Procedural dungeon construction
`player.js`	Player initialization, movement, input handling, and automove
`collision.js`	Collision detection and geometric movement calculations
`renderer.js`	Reserved for future display-related functionality

The current architecture deliberately separates the dungeon's underlying data from its visual representation.

For example, the player character is displayed as `@`, but the underlying dungeon array is not modified to insert that character. The player is rendered as an overlay.

This means that moving away from a staircase reveals the staircase beneath it.

4. Dungeon representation and generation

The dungeon is represented as a two-dimensional JavaScript array: an array of rows, each containing an array of individual map cells.

Current dimensions:

- Height: 60 rows.
- Width: 100 columns.

Current tile representations:

Character	Meaning
`#`	Solid rock
`.`	Excavated floor
`<`	Up staircase
`>`	Down staircase
`@`	Player character (display overlay)

The generator initially fills the entire dungeon with # characters.

Rooms are then excavated by replacing rock with floor tiles.

Room generation

- Room heights are odd numbers between 3 and 9.
- Room widths are odd numbers between 3 and 15.
- Room placement uses center coordinates.
- A five-cell frame is maintained around the map.
- Room overlap is intentional.

The following established variable names must be preserved:

- `center_room_row`
- `center_room_column`
- `top_room_row`
- `bottom_room_row`
- `left_room_column`

- `right_room_column`

These names describe the geometric roles of the values rather than the methods used to calculate them.

Corridor generation

Rooms are connected sequentially using L-shaped corridors.

The current implementation connects the previous room center to the current room center, excavating a horizontal segment followed by a vertical segment.

More sophisticated room-connection algorithms may be considered later, but are not currently necessary.

Staircase generation

The generator collects the coordinates of excavated floor tiles in `floor_coordinates`.

It randomly selects two distinct entries from that collection:

- One becomes the up staircase.
- One becomes the down staircase.

The generator returns an object containing:

- `dungeon`
- `up_stair_pos`

The up staircase position is used to initialize the player.

5. Player initialization and movement

Player initialization is handled by `initialize_player()` in `player.js`.

The player object contains:

- `player_row`
- `player_column`
- `previous_row`
- `previous_column`

The player initially appears at the up staircase.

Keyboard input is handled by `handle_player_input()` using a `keydown` event listener and `event.code`.

Current movement scheme

Numpad key	Direction
8	Up
2	Down

4	Left
6	Right
7	Up-left
9	Up-right
1	Down-left
3	Down-right
5	Wait

The player is rendered by `display_dungeon()` in `main.js`.

The display function iterates through the dungeon array and substitutes `@` whenever the current array coordinates match the player's coordinates.

5.1 Automove

Automove has been implemented in `player.js`.

It is initiated by holding SHIFT while issuing a directional movement command. The player then continues moving automatically according to the geometry of the surrounding dungeon.

The intention is to make automove useful both in relatively open areas and in narrow corridors, including corridors that change direction.

Automove is not simply continuous movement in a fixed direction. It evaluates the surrounding tiles to determine whether movement should continue, turn, or stop.

Open-area behavior

When automove is initiated in an open area, it uses `initial_forward_count` to distinguish that situation.

If all three forward-facing tiles are traversible, automove continues in the original direction. If that condition ceases to hold, automove stops.

Corridor behavior

When automove is initiated in a corridor, it examines the tiles in front of the player and uses their traversibility to determine whether the player can continue straight or should consider an orthogonal turn.

The following established identifiers are involved in the current automove implementation and must be preserved:

- `automove`
- `continue_automove`
- `initial_forward_count`
- `previous_forward_count`
- `forward_tiles`

- `traversible_forward_tiles`
- `direction_row`
- `direction_column`
- `possible_moves`
- `find_forward_tiles`
- `find_orthogonal_moves`

The current implementation uses `previous_forward_count` to compare the forward-facing geometry between successive positions.

Generalized forward-geometry rule

A significant design problem involved determining when automove should stop as a corridor opens into a larger traversible area, without requiring the system to identify rooms, corridors, or junctions as distinct types of spaces.

The solution is based on the arrangement of three forward-facing positions:

- The forward-center tile: immediately ahead of the player.
- The two forward-diagonal tiles: immediately ahead and to either side.
- The second forward-center tile: two steps ahead of the player, in the same direction of travel.

The rule distinguishes the following arrangements:

Forward center	Forward diagonals	Second forward center	Behavior
Open	Both blocked	Either	Continue straight
Open	Exactly one open	Open	Stop
Open	Exactly one open	Blocked	Move forward and continue automoving
Blocked	Both open	Either	Stop

The second forward-center tile is particularly important. It distinguishes a junction-like opening, where automove should stop, from a corner that the player should enter before continuing.

The rule is implemented using the existing forward-tile calculations and `check_collision()`. It relies exclusively on tile geometry and does not classify the surrounding area as a room, corridor, or junction.

Integration with existing automove behavior

The generalized rule is integrated into the existing automove decision-making logic rather than applied unconditionally.

In particular, the condition governing the forward-center and diagonal checks includes:

```
previous_forward_count !== 2 &&
```

```
traversible_forward_tiles === 2 &&
forward_center_open
```

The `previous_forward_count !== 2` restriction is important because ordinary single-wall automovement can also produce two traversible forward-facing tiles. Without this restriction, the generalized stopping rule interferes with continued movement along open wall surfaces.

The existing open-area startup behavior, straight movement, and corridor-turning logic remain in place.

Corridor corners versus room corners

A separate geometric rule distinguishes a genuine corridor bend from a corner along the perimeter of a larger open area.

When the player reaches a position from which an orthogonal turn is being considered, the two tiles flanking the direction of travel are examined at the position immediately preceding the corner.

The relevant coordinates are `player.previous_row` and `player.previous_column`, not the player's current coordinates.

The rule is:

- If both flanking tiles are blocked, the geometry permits a corridor turn.
- If either flanking tile is traversible, automove stops.

The player has already advanced into the corner tile by the time this check is performed. Consequently, stopping does not require another movement.

This distinction is important. An earlier implementation examined the flanking tiles around the player's current position, which was one tile too late and prevented legitimate corridor turns.

Verified automove behavior

The latest implementation has been tested against the following situations:

- Continued movement through straight corridors.
- Automatic turning through genuine corridor bends.
- Stopping after entering a room corner rather than following the room perimeter.
- Stopping at corridor openings into larger expanses.
- Correct stopping at T-junctions.
- Correct stopping where a corridor meets a larger area at a corner, with one corridor wall aligned with one wall of the larger area.
- Continued automovement along ordinary single-wall surfaces.

The T-junction configuration is relatively rare in the current dungeon-generation algorithm, but the user successfully generated and tested one.

The generalized forward-geometry rule has resolved the previously observed stopping errors while preserving the established wall-following and corner-turning behaviors.

Future changes should preserve these behaviors unless a specific defect is identified.

6. Collision detection

Collision detection is implemented in `collision.js` through the exported function:

```
check_collision(dungeon, proposed_row, proposed_column)
```

The function returns a Boolean:

- `true` means the proposed destination can be occupied.
- `false` means the proposed destination cannot be occupied.

The function checks two things:

1. Whether the proposed row and column fall within the dungeon's boundaries.
2. Whether the destination tile is an occupiable tile.

Currently, `.`, `<`, and `>` are considered occupiable.

The boundary checks must occur before attempting to access the proposed dungeon cell. This prevents invalid array indexing.

Additional geometric functions

`collision.js` also contains functions used to evaluate the tiles surrounding a position:

- `check_adjacency()`
- `find_possible_moves()`
- `find_orthogonal_moves()`
- `find_forward_tiles()`

These functions support movement decisions and automove.

In particular:

- `find_forward_tiles()` identifies the three tiles facing the current direction of travel.
- `find_orthogonal_moves()` identifies traversible cardinal neighbors, excluding the player's previous position.
- `check_adjacency()` evaluates the traversible tiles adjacent to a specified position.

These calculations are not limited conceptually to the player's current position. They may eventually be useful for evaluating other positions, including hypothetical or projected positions.

Movement and collision sequence

All eight movement directions follow the same general procedure:

1. Calculate `proposed_row` and `proposed_column`.
2. Pass those coordinates and the dungeon to `check_collision()`.
3. Store the returned Boolean in `valid_destination`.
4. If the destination is valid, assign the proposed coordinates to the player's actual position.
5. Redraw the dungeon.

The distinction between proposed coordinates and actual player coordinates is intentional.

The proposed coordinates describe a possible destination. The collision function evaluates that destination before the player's actual position changes.

Diagonal movement rules

The current implementation checks only the destination tile when moving diagonally.

This permits diagonal corner-cutting when the destination is open, even if one or both intervening cardinal tiles are blocked.

This is a deliberate temporary design decision.

Angband-style and Brogue-style diagonal movement rules have been discussed. No change is currently required.

7. Current development status

The following functionality has been implemented:

Dungeon generation

- Two-dimensional dungeon array initialization.
- Random room dimensions and placement.
- Room excavation.
- Sequential corridor generation.
- Random staircase placement.

Player and display

- Player initialization at the up staircase.
- ASCII dungeon display.
- Player character rendering.
- Eight-direction numpad movement.

- Waiting through numpad 5.

Collision and movement

- Collision detection.
- Dungeon boundary protection.
- Collision checks applied to all eight movement directions.
- Geometric calculations for adjacent, orthogonal, and forward-facing tiles.

Automove

- SHIFT-initiated directional automove.
- Continued movement through straight corridors.
- Automatic turning through corridor bends.
- Stopping at room corners.
- Stopping when corridors open into larger expanses.

The initial dungeon-generation and basic movement implementation phases are complete. Automove has also been implemented, although its behavior should continue to be treated as an established feature that must be protected when introducing subsequent functionality.

8. Future development

The next development phase has not yet been formally established.

Potential future areas include:

- Field of vision and line-of-sight calculations.
- Monster placement and movement.
- Turn-based game logic.
- Combat.
- Additional dungeon features and environmental objects.
- More sophisticated rendering.

These are possibilities, not commitments. Development should proceed one feature at a time.

The existing separation between dungeon construction and display logic should be maintained.

9. Working conventions and version control

The project is maintained independently using Git.

Git is the authoritative record of the source code and its development history.

ChatGPT conversations are used for instruction, design discussion, troubleshooting, and code review. They should not be treated as the authoritative source of the current implementation.

At major development milestones:

1. Commit the working code to Git.
2. Start a new focused ChatGPT conversation when appropriate.
3. Supply the current source files.
4. Use this reference document to establish the project's history, architecture, and design decisions.

The current source files always take precedence over descriptions of earlier implementations in conversation history.

Failed experimental changes should not be retained merely because they might prove useful later. The Git history can preserve earlier working versions without requiring unsuccessful code to remain in the current implementation.

10. Instructions for future ChatGPT conversations

Continue teaching JavaScript incrementally.

Do not assume that because a feature has been implemented, the user necessarily understands every aspect of its implementation.

When introducing new functionality:

- Explain the problem being solved.
- Explain the relevant concepts.
- Discuss the design before presenting substantial code.
- Let the user implement portions themselves whenever practical.
- Review their actual code.
- Correct errors directly and explain their causes.
- Avoid unnecessary changes to working code.

When modifying existing code:

- Request the relevant current code rather than guessing at its contents.
- Preserve all established identifiers.
- Identify exactly what code should be replaced and where.
- Make the smallest change that addresses the identified problem.
- Do not modify unrelated functionality.
- Test the intended behavior and the relevant established behaviors.

- Revert a change completely if it fails to fix its intended problem.

In particular, automove is now an established feature with several interacting geometric rules. Do not casually rewrite or simplify its logic. Understand the purpose of the existing conditions before changing them.

Do not introduce additional conditions simply to compensate for earlier unsuccessful changes. If a generalized geometric rule can solve the problem, prefer that over special cases based on room or corridor classification.

Do not offer praise, compliments, or assessments of the user's input. Keep responses focused on the technical subject.

The central objective remains:

Develop a working roguelike while developing a genuine understanding of the JavaScript used to build it.